

Q.1) Display all the files from current directory whose size is greater than n Bytes Where n is accept

from user [10 Marks]

```
#include <stdio.h>

#include <stdlib.h>

#include <dirent.h>

#include <sys/stat.h>


int main() {

    DIR *dir;

    struct dirent *entry;

    struct stat st;

    long n;


    printf("Enter minimum file size in bytes: ");

    scanf("%ld", &n);


    dir = opendir(".");

    if (!dir) {

        perror("opendir");

        return 1;

    }


    printf("Files larger than %ld bytes:\n", n);

    while ((entry = readdir(dir)) != NULL) {

        if (stat(entry->d_name, &st) == -1) {

            perror(entry->d_name);

            continue;

        }


        if (S_ISREG(st.st_mode) && st.st_size > n) {

            printf("%s\t%ld bytes\n", entry->d_name, st.st_size);

        }

    }

}
```

```
    closedir(dir);  
    return 0;  
}
```

Q.2) Write a C program which creates a child process to run linux/ unix command or any user defined program. The parent process set the signal handler for death of child signal and Alarm signal. If a child process does not complete its execution in 5 second then parent process kills child process

[20 Marks]

```
#include <stdio.h>  
#include <stdlib.h>  
#include <unistd.h>  
#include <signal.h>  
#include <sys/wait.h>
```

```
pid_t child_pid;
```

```
void child_handler(int sig) {  
    int status;  
    wait(&status); // Reap child  
    printf("Child terminated.\n");  
    exit(0);  
}
```

```
void alarm_handler(int sig) {  
    printf("Time limit exceeded. Killing child...\n");  
    kill(child_pid, SIGKILL);  
}
```

```
int main() {  
    struct sigaction sa_child, sa_alarm;  
  
    child_pid = fork();  
    if (child_pid < 0) {
```

```

    perror("fork");
    exit(1);
}

if (child_pid == 0) {
    // Child process: execute a command
    execlp("sleep", "sleep", "10", NULL); // Example command
    perror("execlp"); // if execlp fails
    exit(1);
} else {
    // Parent process
    // Setup SIGCHLD handler
    sa_child.sa_handler = child_handler;
    sigemptyset(&sa_child.sa_mask);
    sa_child.sa_flags = 0;
    sigaction(SIGCHLD, &sa_child, NULL);

    // Setup SIGALRM handler
    sa_alarm.sa_handler = alarm_handler;
    sigemptyset(&sa_alarm.sa_mask);
    sa_alarm.sa_flags = 0;
    sigaction(SIGALRM, &sa_alarm, NULL);

    // Set alarm for 5 seconds
    alarm(5);

    // Wait indefinitely for signals
    while(1) {
        pause();
    }
}

return 0;
}

```